GEDCOM lines *should not*, but *may* contain trailing white space.

GEDCOM lines *should not* have trailing white space because most line values should not have trailing white space.
Things like names, dates and places should have neither leading nor trailing white space, and it is the job of the GEDCOM writer, or rather the application it is part of, to prevent such leading or trailing white space from occurring.

GEDCOM lines *may* contain trailing white space because it cannot always be avoided. The pathological case of a text consisting of lots of white space characters and few newlines forces a GEDCOM writer to create CONC & CONT records that end with white space. A GEDCOM reader must accept those line values as-is, not remove any white space, to correctly reconstitute the original text.

Trailing White Space

GEDCOM lines *must not* not end with superfluous trailing white space.
GEDCOM lines *may* contain significant trailing white space.

A GEDCOM writer, or rather the system it is part of, must ensure that it does not write superfluous white space, but only significant white space. For many fields, applications should automatically strip any leading and trailing white space upon user input. Developers should include warnings about leading and trailing white space as part of the consistency checks.

A GEDCOM reader must import the lines values presented by a GEDCOM file as they are. A GEDCOM reader must treat every part of a line value, including all white space, as significant. **A GEDCOM reader *must* import every line values as-is, *must not* not strip trailing white space.**

Leading White Space

The rules for leading and trailing white space are not really rules about GEDCOM lines, but about GEDCOM lines values. Line values *should not*, but *may* contain trailing white space. The same goes for leading white space: Line values *should not*, but *may* contain leading white space.

A line value may start with leading white space. That CONC line values starts with leading white space is often deliberate. A GEDCOM reader must import the lines values presented by a GEDCOM file as they are. A GEDCOM reader must treat every part of a line value, including all white space, as significant. **A GEDCOM reader *must* import every line values as-is, *must not* not strip leading white space.**

CONC & CONT

GEDCOM Grammar

The CONC & CONT records are defined as part of the GEDCOM grammar. They are always available for use with any line value of any record in any GEDCOM form.

The CONC & CONT records must not be used with records defined in the GEDCOM grammar. They may only be used with records defined in a GEDCOM form.
There is no such thing as CONC or CONT records subordinate to CONC or CONT records.

The CONC & CONT record always appear as immediate subrecords of a GEDCOM form record. There is no such thing as a top-level CONC or CONT record.

GEDCOM 5.5.5 Rule

The GEDCOM 5.5.5 specification introduces a new rule

While it is allowed to use CONC records for any line value, GEDCOM writers *should* not use them needlessly. GEDCOM writers *should not* use CONC records for line values with a maximum length of 248 code unit or less.

GEDCOM 5.5.5 introduces one exception to the rule that CONC & CONT records may be used with any records; the CONC & CONT records *must not* be used in the GEDCOM header.

Logical versus Physical Line Value

This text distinguishes between logical line values and physical line values. The GEDCOM grammar defines physical line values, and a GEDCOM form defines logical values:

- The GEDCOM grammar defines physical line values. A physical line value may be at most 248 code units long, and *must not* contain any carriage return (CR) or line feed (LF) character.
- A GEDCOM form defines logical values. A single logical line value may be a long text consisting of multiple lines. A single logical line value may be up to 32.765 code units long, and may contain newlines.

The CONC & CONT records exist to bridge the differences. The CONC & CONT records enable support of logical line values despite the limitations of the physical lines values.

Line Terminator versus Newline

This text refers to the line break at end of a GEDCOM line as a *line terminator*, and a line break inside a logical line value as a *newline*. This convention should make the text easier to understand, and allows making the following point, which is hard to make otherwise: the line terminator used for GEDCOM lines and the newline used within logical line values need not be the same. For example, a web application running on a Unix server would typically use line feed (LF) as its newline inside the logical line value, and the carriage return/line feed (CR/LF) combination as the line terminator for its GEDCOM lines.

CONC

The CONC (concatenation) record allows a single logical line value to split over multiple GEDCOM lines. This is needed because a GEDCOM line must not be more than 255 character long, and GEDCOM must still be able to handle logical line values longer than that.
It is called the concatenation record because a GEDCOM reader must concatenate the physical line value of a CONC record to the logical line value of the superior record to reconstitute the original logical line value.

CONT

The CONT (continuation) record is how GEDCOM supports multiple lines *within* a logical line value. This is needed because a logical line value may contain newlines, but a physical line value must not contain any carriage return (CR) or line feed (LF) characters. It is called the continuation record because a GEDCOM reader must continue on a new line (by reinserting the newline that prompted the GEDCOM writer to use a CONT record) before appending the physical line value of the CONC record to the physical line value of the superior record.

Usage

Usage of the CONC & CONT records is simple. A GEDCOM writer splits a logical line value into parts as needed, and writes those parts as the physical line values of a subordinate CONC or CONT record. A GEDCOM reader reconstitutes the logical line value by recombining the line value of the record with the line values of the CONC & CONT subrecords.

CONC Usage

A GEDCOM writer creates another CONC record whenever the remaining logical line value is too long for the current physical record.
A GEDCOM reader simply appends the physical line value of the CONT record to the logical line record is it reconstituting, *without adding anything in between*.

CONT Usage

A GEDCOM writer creates a new CONT record whenever it encounters a newline, then continues writing the rest of the logical line value.
When a GEDCOM reader encounter a CONT record, it appends a newline to the logical line value it is reconstituting before appending the physical line value of the CONT record.

Character and Code Point Boundaries

A GEDCOM writer cannot split a line wherever it likes, and certainly not at some fixed number of code units. After every split, the two resulting parts must both be valid string for the encoding used.

Text-splitting must respect *code point* boundaries. Many Unicode code points require multiple code units, not only when UTF-8 is used, but also when UTF-16 is used. Text-splitting *must not* split at just any code unit, as that might split the text *inside* a code point. Text-splitting must occur *between* code points.

Text-splitting must respect *character* boundaries. Unicode support characters that consists of multiple code points. Text-splitting *must not* split at just any code point, as that might split the text *inside* a character. Text-splitting must occur *between* characters.

Developers *must not* simply split a line value at a fixed number of code units, or some fixed number of code points, they must ensure that they split between characters. Developers need not implement their own string manipulation routines to get this right. Developers can and should avoid string handling problems by taking advantage of the string routines offered by the their programming language's library, framework, platform or operating system. These routines know how to split Unicode strings.

The ANSEL character set allows characters consisting of multiple code points too. There are no library routines that perform correct splitting of ANSEL strings, but that does not matter, as there is no need to split ANSEL strings anymore. The *GEDCOM 5.5.1 Annotated Edition* recommends using Unicode, the GEDCOM 5.5.5 specification demands Unicode.
Developers do not need to write ANSEL at all, and only to read ANSEL if they want to read GEDCOM 5.5.1 and earlier files, including ANSEL GEDCOM files.

Practical Problems

Conceptually, using CONC & CONT is simple. Historically, there are two practical problems:

- The GEDCOM writer must figure out where to break the logical line value.
- The GEDCOM reader must decide what to do with leading and trailing white space.

Trailing White Space

The GEDCOM specification does not forbid trailing white space, but for most records, trailing white space is meaningless. Historically, many GEDOM readers strip trailing white space from records.
That is problematic, as the trailing white space on a CONC or CONT record may be the space between words that would be erroneously concatenated if that space were lost.

That is why GEDCOM 5.5.1 requires that a split be done either in the middle of word, or just

before a space. When the split occurs just before a space, that space becomes the first character of the line value on the next line. Such leading white space is considerably less likely to be lost than trailing white space. It might still get lost because some GEDCOM readers skip all white space after a tag, instead of skipping just one space as they should.

The CONC or CONT annotation in the *GEDCOM 5.5.1 Annotated Edition* discusses the problems with and solutions for GEDCOM 5.5.1 and earlier in some detail.

GEDCOM 5.5.5 specifies the simplest possible solution to all the CONC or CONT complications. It is straightforward rule that GEDCOM 5.5.5 readers must follow anyway: a GEDCOM reader must import line values correctly.
A GEDCOM reader must import the lines values presented by a GEDCOM file as they are. A GEDCOM reader must treat every part of a line value, including all white space, as significant. A GEDCOM reader *must not* trim leading or trailing white space from line values. A GEDCOM reader must simply import the line values as-is, including all leading and trailing white space.

GEDCOM 5.5.5 Writer Best Practice

- Use operating system or programming library functions to make sure you split between characters, not inside characters
- Otherwise, split wherever is convenient; this GEDCOM file will be read by a GEDCOM 5.5.5 or later reader

GEDCOM 5.5.5 Reader Rules

- import each line value *as-is*
- *do not* trim trailing white space from any GEDCOM line or line value
- *do not* trim leading white space from any line value

References

- GEDCOM CONC and CONT
- Gaenovium Presentations: Louis Kessler: Reading Wrong GEDCOM Right
- Behold blog 2010-01-10: CONC Me on the Head

Symbols Used with Basic GEDCOM Language and GEDCOM forms

The following symbols are used definitions of the Basic GEDCOM Language and GEDCOM forms:

<<double_angle bracket>>
The name within the brackets is a subordinate structure of one or more GEDCOM records (and subrecords) that is to be substituted in place of the line containing the enclosing double angle brackets.

<Single_angle bracket>
The name within the brackets is a basic value type, also known as a primitive.

{braces}
Indicates the minimum and maximum occurrences allowed for this structure or line—{Minimum:Maximum}. Note that minimum and maximum occurrence limits are defined relative to the enclosing superior line. This means that a required line (minimum = 1) *is not required if the optional enclosing superior line is not present*. Similarly, a line occurring only once (maximum = 1) may occur multiple times as long as each occurs only once under its own multiple-occurring superior line.

For textual line values, the numbers specify the minimum and maximum length of the *logical line value*. Long logical line values may require the use of CONC & CONT records

to keep all physical line values below the maximum of 255 code units.

When the minimum and maximum are identical, a shorthand is used; the definition simply gives the size, instead of repeating it with a colon in between, so {Size=3} instead of {Size=3:3}.

[Square brackets]
Indicates a choice of one or more options— **[**Choice of**]**.

vertical | bar
Separates the multiple choices, for example [Choice 1 | Choice 2].

n level number
A level number which assumes the level number of the line which referenced the substructure name.

+1, +2 ...
A **+1** level number is 1 greater than the level number assumed by the superior **n** level. A **+2** level number is 2 greater, and so forth.

0xHH
Indicates an allowable hexadecimal character value where HH is that value, for example, 0x20 (decimal 32) indicates the space character.

Chapter 1 Part II: Basic GEDCOM Language

The Basic GEDCOM Language is the GEDCOM language shared by all GEDCOM forms. It defines essential GEDCOM records and the overall structure of a GEDCOM file.

Basic GEDCOM

Four Record Types

The GEDCOM Language defines four record types (the header record has a few subrecords):

- **HEAD**: a *header* record to provide basic information about a GEDCOM file.
- **TRLR**: a *trailer* record to signal the end of a GEDCOM file.
- **CONC**: a *concatenation* record to support long line values.
- **CONT**: a *continuation* record to support newlines.

The header record provides just the essentials: GEDCOM version, character encoding, GEDCOM form and form version. The GEDCOM form may extend the header with additional information of relevance to the form.

The trailer record is technically superfluous, but its presence reassuring; its absence allows reporting of incomplete GEDCOM files. It was necessary for when GEDCOM still supported multi-volume GEDCOM files.

The CONC & CONT address limitations of the GEDCOM grammar. They are defined as part of the GEDCOM Language for used with record defined in GEDCOM forms.

GEDCOM File High-Level Structure

A GEDCOM file starts with a header, ends with trailer record, and has zero or more GEDCOM form-specific record in between. A specific GEDCOM may demand a minimum number of form records, the GEDCOM language does not.

GEDCOM_FILE:=

This is the high-level structure of a GEDCOM file: a GEDCOM file starts with a GEDCOM header, which may be extended by the GEDCOM form. The header is followed by any number of form-specific records, and the file is terminated with a trailer record.
The header and trailer record are mandatory, the header extension and form records are *conceptually* optional.

0 <<GEDCOM_HEADER>>	{1:1}	p. 50
+1 <<GEDCOM_FORM_HEADER_EXTENSION>>	{1:1}	p. 49
0 <<FORM_RECORDS>>	{1:1}	p. 48
0 <<GEDCOM_TRAILER>>	{1:1}	p. 51

The above definition demands the presence of <<GEDCOM_FORM_HEADER_EXTENSION>>, to leave the actual demand up to each GEDCOM form. If the <<GEDCOM_FORM_HEADER_EXTENSION>> were optional in this definition, no GEDCOM form could demand the presence of its extensions. A GEDCOM form can go without any GEDCOM header extensions by defining its <<GEDCOM_FORM_HEADER_EXTENSION>> to be empty.

Similarly, the above definition demands the presence of <<FORM_RECORDS>>, to leave the actual demand up to each GEDCOM form.

GEDCOM Record Definitions

CHARACTER_ENCODING:= **{Size=5|7}**

[UTF-8 | UNICODE | ANSEL | ASCII]

A coded value (exhaustive enumeration) that represents the character set and encoding to be used to interpret this data.
The name of the UNICODE value is confusing, it should have been called UTF-16.

The ASCII and ANSEL character sets are obsolete; the ASCII and ANSEL encoding are *not* legal in GEDCOM 5.5.5.
Unicode-based systems creating GEDCOM 5.5.5 files *must not* offer users the ability to export using non-Unicode encodings, as such exports are practically sure to lose information.
It is strongly recommended that applications support *import* of ASCII and ANSEL GEDCOM files for GEDCOM 5.5.1 and earlier.

All GEDCOM 5.5.5 files *must* use Unicode, using either the UTF-8 or UTF-16 encoding.
All GEDCOM 5.5.5 files, UTF-8 as well as UTF-16 GEDCOM files, *must* start with a Byte Order Mark (BOM).

Western applications should default to using the UTF-8 encoding, Eastern application should default to using UTF-16 encoding.
Application *need not* offers user a choice between these two options.

A GEDCOM 5.5.5 reader *must not* tolerate anything but the UTF-8 and UTF-16 encodings, but *must* reject files using any other encoding as not-GEDCOM, specifically as not even having a valid GEDCOM header.
A GEDCOM 5.5.5 reader *must not* accept a GEDCOM file without a BOM, but *must reject* such files file as *not* GEDCOM 5.5.5, with a message such as “ostensible GEDCOM 5.5.5 file lacks Byte Order Mark”.

A developer may choose to always use just one legal Unicode encoding on export of GEDCOM 5.5.5, but *must* support import of all legal Unicode encodings. A GEDCOM 5.5.5 reader *must* support both UTF-8 and UTF-16 GEDCOM files. The GEDCOM import must also support both Little-Endian and Big-Endian UTF-16.

Unicode applications generally UTF-16 internally, so adding UTF-16 support is fairly straightforward. Support for both Little-Endian and Big-Endian UTF-16 takes no more than a byte swap for every word.
However, even UTF-16 text *must not* be imported with a simple copy operation. Different operating systems use different Unicode normal forms. For example, Windows uses Normal Form C, while Mac OS X uses Normal Form D. Applications should always use library, framework or operating system string routines, to ensure their Unicode text has the proper normal form.

Code-page applications must not use GEDCOM 5.5.5, but must continue to use GEDCOM 5.5.1, and should default their GEDCOM export to UTF-8 instead of ANSEL.

GEDCOM 5.5.5 Character Encoding Rules

GEDCOM 5.5.5 Writer

- Use UTF-8 and UTF-16 only
- Western applications should default to UTF-8
- Eastern applications should default to UTF-16

GEDCOM 5.5.5 Reader

- demand that file starts with a Byte Order Mark (BOM)
- demand that the encoding is either UTF-8 or UTF-16
- must support both UTF-8 and UTF-16 GEDCOM files
- must support both Little-Endian and Big-Endian UTF-16 GEDCOM files
- reject files using anything else as not-GEDCOM, not even a valid GEDCOM header

GEDCOM 5.5.1 Detection

The introduction of GEDCOM 5.5.5 does not change the detection of GEDCOM 5.5.1 files. GEDCOM 5.5.5 files must be identified as GEDCOM 5.5.5 files; if they are not identified as GEDCOM 5.5.5, they aren't GEDCOM 5.5.5 files.
Any file that claims to be a GEDCOM 5.5 file is either a GEDCOM 5.5.1 file (most likely) or an actual GEDCOM 5.5 file. It is never a GEDCOM 5.5.5 file.

Any GEDCOM file that claims to be a GEDCOM 5.5 file but uses the UTF-8 encoding must be recognised as a GEDCOM 5.5.1 file with a truncated version number.

References

- GEDCOM Character Encodings
- GEDCOM Version detection

FORM_RECORDS:= {1:1}

GEDCOM-form specific records. These top-level records are defined by the GEDCOM form specified in GEDCOM_FORM.

Notice that the <<GEDCOM_FILE>> definition does not contain a <<FORM_RECORD>> (singular) that is allowed to occur many times, but a <<FORM_RECORDS>> (plural) that is allowed to occur just one.
This allows a GEDCOM form to not only define GEDCOM records types, but their order as well.

GEDCOM_FORM:= {Size=1:20}

A value that identifies the GEDCOM form used in this GEDCOM file. The value must be alphanumerical string. This string is case-sensitive.

The GEDCOM grammar does not define any GEDCOM form. The identifier for a GEDCOM form is defined by the specification for that GEDCOM form.
Chapter 2 of this specification defines the Lineage-Linked Form, identified by the value `LINEAGE-LINKED`.

Validity Check

A GEDCOM 5.5.5 reader *must* check the specified GEDCOM_FORM. It does so by performing a case-sensitive check against the list of known GEDCOM forms it supports. A GEDCOM reader *must not* attempt to correct for casing or spelling errors in the GEDCOM form.

A GEDCOM reader must reject any file without a <GEDCOM_FORM> as not a GEDCOM file. *A GEDCOM reader may only try to process a GEDCOM file it if recognises and supports the GEDCOM form used.* If a GEDCOM reader does not recognise the GEDCOM form, it *must* abort with a message such as “Unrecognised GEDCOM form”. If a GEDCOM reader recognises the GEDCOM form, but does not support it, it *must* abort with message such as “Unsupported GEDCOM form”.

GEDCOM_FORM_HEADER_EXTENSION:= **{1:1}**

Zero or more additional HEAD subrecords, as defined by the GEDCOM form specified in HEAD.GEDC.FORM.

GEDCOM Records

GEDCOM_VERSION_NUMBER:= **{Size=3:11}**

MMM + dot + mmm [+ dot + rrr]

where:

MMM = 1 through 3 digits; the major version number
mmm = 1 through 3 digits; the minor version number
rrr = 1 through 3 digits; the revision number
dot = (2E), the full stop character

The version number of the specification used.

Version Number is a Contract

There are two GEDCOM version numbers; one for the GEDCOM specification, and one for the GEDCOM form used.
Both versions numbers are contracts; a promise that everything in the GEDCOM file conforms to that GEDCOM specification and that GEDCOM form, that the GEDCOM file only contains records that are legal according to those specifications.

Three Values

A GEDCOM version consists of at least two and at most three dot-separated values in *major.minor.revision* format. The three parts of the *major.minor.revision* format are:

major = the major version number; starts at 0 (zero).
minor = the minor version number; starts at 0 for each new *major* version.
revision = the revision number; starts at 0 for each new *major.minor* version.

Dot-Separated Values

The full stops used in version numbers are known as dots, but pronounced as “point”. A version number never begins or ends with a dot, it always begins and ends with a digit. Values used must appear in the order shown. The major and minor version number are both mandatory. The minor version number must included, even if it is zero.
The revision number may be left off when it zero. When missing, it must be assumed to be zero.

The *major* version number signals a major new release, often including major new features or breaking changes. The *minor* version number version number signals a significant update of the same release, often including some significant new features, but no breaking changes. The *revision* number signals a lesser update, typically including only minor changes.

Each value consists at most 3 digits. Zeroes are allowed, leading zeroes are not; `0.99` is a valid version number, and so is `1.0`, but `1.01` is not. Trailing zeroes must not be left out and aren't implied; version `2.1` and version `2.10` are two different versions, and version `2.1` comes before version `2.9`. The minimum version number is `0.1`. The maximum version number is `999.999.999`.

Older GEDCOM Versions

It is strongly recommended that systems that continue to export GEDCOM 5.5.1 or earlier

files in addition to GEDCOM 5.5.5 files, comply with the GEDCOM 5.5.5 version number rules for all supported versions.

References

- Truncated GEDCOM Version
- GEDCOM Version Detection

GEDCOM_HEADER:=

U+FEFF (Byte Order Mark)

n **HEAD**	{1:1}	
+1 **GEDC**	{1:1}	
+2 **VERS** <GEDCOM_VERSION_NUMBER>	{1:1}	p. 49
+2 **FORM** <GEDCOM_FORM>	{1:1}	p. 48
+3 **VERS** <GEDCOM_VERSION_NUMBER>	{:1}	p. 49
+1 **CHAR** <CHARACTER_ENCODING>	{1:1}	p. 47

...

Essential Information

This is the definition of a basic GEDCOM header.
The basic GEDCOM header provides just the essential information; everything that's needed for a system to figure out whether it can read this GEDCOM file, no more, no less.
Every GEDCOM file must start with this header.
The continuation dots indicate that the header may be extended by the GEDCOM form.
The GEDCOM header is always preceded by the Byte Order Mark (BOM), Unicode code point U+FEFF.

Valid Header

It is important that GEDCOM 5.5.5 writers produce correct GEDCOM files, and especially important that each file contains a valid GEDCOM header.
A GEDCOM 5.5.5 reader must demand a flawless GEDCOM header; it *must not* process a GEDCOM file if the header is invalid, but must issue a fatal error and abort.

No Modifications

Developers *must not* modify or extend this record in any way.
No additional records may occur, not even CONC or CONT records.

HEAD.CHAR Record required

The HEAD.CHAR record is required.
GEDCOM 5.5.5 demands that GEDCOM files use Unicode, allows only the UTF-8 and UTF-16 encodings, and demands that each GEDCOM file starts with a Byte Order Mark (BOM). That Byte Order Mark already tells the GEDCOM what the encoding of the GEDCOM file is. So, technically, the HEAD.CHAR record is superfluous now. The HEAD.CHAR record remains mandatory for maximum compatibility with GEDCOM 5.5.1 and earlier.

GEDCOM Form Header Extension

GEDCOM forms may extend the Basic GEDCOM header with additional records to create a form-specific header. The GEDCOM form can only *extend* the GEDCOM header; all the additional records must come after the records of the basic GEDCOM header. Use of CONC

nor CONT records is allowed there, but strongly discommended.

Explicit Support required

A GEDCOM writer *must* provide correct **GEDC.VERS** and **GEDC.FORM** values. Systems which process GEDCOM files *must* check theses values, and may only try to read GEDCOM versions and forms it explicitly supports. For GEDCOM versions or forms it does not support, the system must state so and abort, it *must not* try to import a GEDCOM version or form it does not explicitly support.

Form Version

GEDCOM 5.5.5 introduces the HEAD.GEDC.FORM.VERS, a separate version number for the GEDCOM form, in addition to HEAD.GEDC.VERS, the general GEDCOM version number. HEAD.GEDC.FORM.VERS record is mandatory.
Systems *must not* try to read GEDCOM 5.5.5 files if they do not explicitly support GEDCOM 5.5.5, but if a system that only knows about GEDCOM 5.5.1 and earlier tries to do so anyway, and follows the rule that unrecognised records may be ignored, it will probably ignore the HEAD.GEDC.FORM.VERS record.

GEDCOM_TRAILER:=

0 **TRLR** {1:1}

The GEDCOM trailer marks the end of a GEDCOM file.
The TRLR record has neither a line value nor subrecords.
The TRLR record ends with a line terminator *just like any other record*. That terminator must not be absent, and the TRLR record must not be followed by anything else.

GEDCOM 5.5.1 still needed the TRLR record in support of multi-volume GEDCOM files, GEDCOM 5.5.5 no longer allows multi-volume GEDCOM files.
Technically, the TRLR *is* superfluous now, but it remains for backward compatibility and because it's is reassuring; the TRLR record confirms that the end of a complete GEDCOM file has been reached.

GEDCOM Tag Definitions

CHAR {CHARACTER_ENCODING}:=

The character set and encoding used in this GEDCOM file.

CONC {CONCATENATION}:=

Additional text for the superior line value. A CONC record without a line value is a fatal syntax error.

The CONC record exists to allow logical line values longer than the maximum physical line value. A GEDCOM writer must split long line values into parts and use subordinate CONC records to record the additional parts. A GEDCOM reader must append CONC line value must be appended to logical line value it is reconstituting for the superior record.
The CONC & CONT section discusses CONC & CONT usage in detail.

CONT {CONTINUATION}:=

A newline and usually some additional text as well for the superior line value. A CONT record without a line value is *not* an error, but the GEDCOM encoding of an empty line.

The CONT record exists to allow logical line values to contain newlines, while physical lines must not contain newlines. A GEDCOM writer must split a line values at a newline, and then continue writing the value using a subordinate CONT record. A GEDCOM reader must append a newline to the logical line value it is reconstituting for the superior record

before appending the CONT line value itself.
The CONC & CONT section discusses CONC & CONT usage in detail.

FORM {FORM}:=

The name of the *GEDCOM form* used in this GEDCOM file.
The GEDCOM specification defines just one GEDCOM form: `LINEAGE-LINKED`.

GEDC {GEDCOM}:=

Information about the use of GEDCOM in a GEDCOM file.

HEAD {HEADER}:=

Information about the entire GEDCOM file.

TRLR {TRAILER}:=

A top-level record that marks the end of the GEDCOM file.

VERS {VERSION}:=

Version number.

Chapter 2 Lineage-Linked Form

Introduction

The Lineage-Linked Form defined in this chapter is a GEDCOM form based on the general framework of the Basic GEDCOM Language defined in Chapter 1.

The Lineage-Linked Form is a couple-centric GEDCOM form; the family group record for a couple and their children is placed centrally in this design. This GEDCOM form is called lineage-linked because it pertains to individuals linked in parent-child relationships across multiple generations.
This chapter describes the specific tag, value, and pointer combinations used for exchanging genealogical information in this format. The chapter also addresses specific compatibility issues pertaining to previous Lineage-Linked Form releases and contains a simple lineage-linked GEDCOM file example.

Organisation

The basic description of the **Lineage-Linked GEDCOM Form** is presented in the following four major sections:

! "Lineage-Linked Form definition", page 55
! "Top-Level Records of the Lineage-Linked Form", page 56
! "Subrecords of the Lineage-Linked Form, page 64
! "Primitive elements of the Lineage-Linked Form", page 75

The definition of the tags used in defining the lineage-linked structures are contained in Appendix A.

Lineage-Linked Form Usage Conventions

! The order in which GEDCOM lines are written to a GEDCOM file is controlled by the context (level and parent record). The Lineage-Linked syntax may demand that particular records be in a particular order, e.g. the submitter record must follow directly after the GEDCOM header, but otherwise, the order of different records within the same context is not significant.
It is possible for the same record type to occur multiple time within the same context. This happens in two situations: events that may happen more than once (e.g. burial), and events that happen just once, but for which there are multiple possible conflicting values (e.g. birth date). In the latter case, the order of the records is interpreted as the submitter's preference. *The most preferred value being the first with the least preferred data listed in subsequent lines by order of decreasing preference.* For example, a researcher who discovers conflicting evidence about a person's birth event would list the most credible information first and the least credible or least preferred items last.

! Systems that support multiple fields or structures should allow their users to indicate their preference opinion. Systems that only store single value structures should use the preferred information (the first occurrence listed) and store the remaining information as an exception, preferably *within an appropriate NOTE field* or in some way that the user has ready access to the less-preferred data when viewing the record.

! Conflicting event dates and places should be represented by placing them in separate event structures with appropriate source citations rather than by placing them under the same enclosing event.

! The Lineage-Linked GEDCOM Form uses the TYPE tag to classify its superior tag for the viewer. The value portion given by the TYPE tag is not intended to inform a computer program how to process the data, unless there is a list of standardised or controlled line_value choices given by the definition of the line value in this standard. The difference between an uncontrolled line value and a note value is that displaying systems should always display the *type value* when they display the data from the associated context. This gives the

user flexibility in further defining information in a compatible GEDCOM context and the reader to understand events or facts which have not been classified by a specific tag. For example:

1 EVEN
2 TYPE Awarded BSA Eagle Rank
2 DATE 1980

! All controlled line_value choices (enumerated values) are case-insensitive. The values "Feb", "FEB" and "feb" are considered equal.
A GEDCOM reader must convert all such values to all-uppercase or all-lowercase prior to comparing to internally defined values.

GEDCOM tags are case-sensitive. All the tags of the Lineage-Linked Form are UPPERCASE.
User-defined tags should be UPPERCASE as well.

! All GEDCOM lines but the last one have either a **value** or a **pointer** unless the line contains subordinate GEDCOM lines. The Lineage-linked form **does not allow** a GEDCOM line with both a value and a pointer on the same line.

For anything but the TRLR record, the presence of only a level number and a tag, without any value or any subrecords, is a syntax error. For example, the way to assert that a death happened at an unknown time and place is not the line **1 DEAT** without subrecords, but the line **1 DEAT Y** without subrecords.

User-defined Records

Any record that isn't part of the Basic GEDCOM Language or the GEDCOM form used, is assumed to be a user-defined record. User-defined records make use of user-defined tags. All user-defined tags must start with an underscore (_).
Tags may not contain additional underscores. Developers who use tags with additional underscores in GEDCOM 5.5.1 or earlier may continue to do so, but must leave out these underscores in GEDCOM 5.5.5 or later.

additional rules regarding predefined tags

- It is illegal to use any user-defined tags when predefined tags are sufficient.
- It is illegal to define a user-defined tags that equals a predefined tag with an underscore in front.
- It is also illegal to try and "bring back" a tag that has been obsoleted in GEDCOM 5.5.5 or later by prefixing it with an underscore.
- It is legal to support a new record in earlier versions of GEDCOM by using the new tag prefixed with an underscore, but only in support of that new record in older versions.

So, it is illegal to use `_TOWN` to record the city within an address, because we already have the CITY record to do so. It is illegal to use `_CITY` for anything, because the `CITY` tag already exists. It is illegal to `_SSN` in GEDCOM 5.5.5 or later, because GEDCOM 5.5.5 obsoleted SSN. It is legal to use `_EMAIL` in GEDCOM 5.5, to support the EMAIL record introduced in GEDCOM 5.5.1.
The list of regular tags that may not be turned into user-defined by prefixing them with an underscore consists of all the predefined tags, and the tags obsoleted in GEDCOM 5.5.5.

End User-Defined Tags

There are developer-defined records and truly end user-defined records. The GEDCOM specification does not distinguish between those two categories. When the GEDCOM specification mentions user-defined records or tags, that generally means developer- and product-specific records and tags, but it is not uncommon for genealogy applications to allow users to define their own records and tags. It is up to applications to keep all user-defined tags legal by

making sure they all start with an underscore. Applications should keep all developer-defined tags all-uppercase, and use all-lowercase for truly end-user defined tags. This allows third parties to easily distinguish between these two different categories of so-called user-defined tags.

tag interpretation

The interpretation of developer-defined tags depends on the GEDCOM dialect as specified by HEAD.DEST. The interpretation of truly *end user*-defined tags is anyone's guess.

The HEAD.SOUR line value identifies the product that created the GEDCOM file - and that is all it identifies. The GEDCOM dialect used within a GEDCOM file, and thus the interpretation of GEDCOM extensions, is indicated by the HEAD.DEST line value.
In a typical GEDCOM file, the HEAD.DEST is equal to the HEAD.SOUR value.

Reading Third-Party Extensions

Although it is perfectly legal for any genealogy application to ignore so-called user-defined tags, that generally isn't what users want or expect. In practice, many genealogy software developers try to support the most important GEDCOM extensions used by other developers on GEDCOM import, as that provides a better user experience than ignoring them. Genealogy software developers with products that understand GEDCOM extensions used by other products often tout the product's ability to import GEDCOM files created by those products as a selling point.

Developers can and should improve third-party support for their extensions by publicly documenting them.

References

- GEDCOM SOUR and DEST
- GEDCOM System Identifiers
- Behold blog 2011-11-21: A Plethora of Extra GEDCOM Tags

Lineage-Linked Form Definition

The Lineage-Linked Form is a GEDCOM form. A file using the Lineage-Linked Form is known as a lineage-linked GEDCOM file.

A Lineage-Linked GEDCOM File

- is identified by the **<<GEDCOM_FORM>>** value `LINEAGE-LINKED`
- extends the Basic GEDCOM header with the **<<LINEAGE_LINKED_HEADER_EXTENSION>>**
- contains zero or more **<<LINEAGE_LINKED_RECORD>>**
- contains exactly one **<<SUBMITTER_RECORD>>** that follows directly after the GEDCOM header

<<GEDCOM_FORM>>

The definition of the Lineage-Linked Form builds on the definitions of the Basic GEDCOM Language. The high-level Lineage-Linked Form definitions are as follows:

GEDCOM_FORM:= **{Size=14:20}**

[LINEAGE-LINKED]
The line value `LINEAGE-LINKED` identifies the Lineage-Linked form.

A GEDCOM 5.5.5 reader *must* check the specified <GEDCOM_FORM> for validity. A

GEDCOM 5.5.5 reader must reject any file without a <GEDCOM_FORM> as not a GEDCOM file. If the <GEDCOM_FORM> isn't LINEAGE-LINKED or another supported GEDCOM form, the GEDCOM reader *must not* try to process the file, but must refuse to process the file with an *unsupported GEDCOM form* message.
Many GEDCOM 5.5 and GEDCOM 5.5.1 readers correct for known `LINEAGE-LINKED` spelling errors known to occur in GEDCOM 5.5 and 5.5.1 files. A GEDCOM 5.5.5 reader *need not* and *must not* do so.

GEDCOM_FORM_HEADER_EXTENSION:= **{1:1}**

n <<LINEAGE_LINKED_HEADER_EXTENSION>>	{1:1}	p. 57

HEAD subrecords specific to the Lineage-Linked Form.

FORM_RECORDS:=

0 <<SUBMITTER_RECORD>>	{1:1}	p. 63
0 <<LINEAGE_LINKED_RECORD>>	{0:M}	p. 58

A Lineage-Linked files consists of zero or more lineage-linked records, preceded by a submitter record. Thus, the submitter record follows directly after the GEDCOM header.

Lineage-Linked GEDCOM File

When you merge the <<GEDCOM_HEADER>> and the high-level Lineage-Linked Form definitions into the <<**GEDCOM_FILE**>> definition, you get the following Lineage-Linked GEDCOM file definition.

LINEAGE_LINKED_GEDCOM_FILE:=

U+FEFF (Byte Order Mark)		
n HEAD	{1:1}	
+1 **GEDC**	{1:1}	
+2 **VERS** <GEDCOM_VERSION_NUMBER>	{1:1}	p. 49
+2 **FORM** LINEAGE-LINKED	{1:1}	
+3 **VERS** <GEDCOM_VERSION_NUMBER>	{1:1}	p. 49
+1 **CHAR** <CHARACTER_ENCODING>	{1:1}	p. 47
+1 <<LINEAGE_LINKED_HEADER_EXTENSION>>	{1:1}	p. 57
0 <<SUBMITTER_RECORD>>	{1:1}	p. 63
0 <<LINEAGE_LINKED_RECORD>>	{0:M}	p. 58
0 <<GEDCOM_TRAILER>>	{1:1}	p. 51

Top-Level Records of the Lineage-Linked Form

LINEAGE_LINKED_GEDCOM_FILE:=

U+FEFF (Byte Order Mark)		
n HEAD	{1:1}	
+1 **GEDC**	{1:1}	
+2 **VERS** <GEDCOM_VERSION_NUMBER>	{1:1}	p. 49
+2 **FORM** LINEAGE-LINKED	{1:1}	
+3 **VERS** <GEDCOM_VERSION_NUMBER>	{1:1}	p. 49
+1 **CHAR** <CHARACTER_ENCODING>	{1:1}	p. 47
+1 <<LINEAGE_LINKED_HEADER_EXTENSION>>	{1:1}	p. 57
0 <<SUBMITTER_RECORD>>	{1:1}	p. 63
0 <<LINEAGE_LINKED_RECORD>>	{0:M}	p. 58
0 <<GEDCOM_TRAILER>>	{1:1}	p. 51

Submitter Record

A Lineage-Linked GEDCOM file consist of a header record, a trailer record, and other records in between. This top-level definition of the lineage-linked form includes the mandatory submitter record explicitly because it must follow the header record directly. This demand is made to avoid breaking existing GEDCOM parsers, which expect the submitter record to be there.

Version Number

For Lineage-Linked GEDCOM files based on the GEDCOM 5.5.5 specification, the GEDCOM version is `5.5.5` and the Lineage-Linked Form version is `5.5.5` too.

LINEAGE_LINKED_HEADER_EXTENSION:=

n **DEST**<RECEIVING_SYSTEM_NAME>	{0:1}	p. 103
n **SOUR** <SYSTEM_ID>	{1:1}	p. 106
+1 VERS <PRODUCT_VERSION_NUMBER>	{0:1}	p. 102
+1 NAME <NAME_OF_PRODUCT>	{0:1}	p. 96
+1 CORP <NAME_OF_BUSINESS>	{0:1}	p. 96
+2 <<ADDRESS_STRUCTURE>>	{0:1}	p. 64
+1 DATA <NAME_OF_SOURCE_DATA>	{0:1}	p. 96
+2 DATE <PUBLICATION_DATE>	{0:1)	p. 103
+2 COPR <COPYRIGHT_SOURCE_DATA>	{0:1)	p. 79
n DATE <FILE_CREATION_DATE>	{0:1}	p. 93
+1 TIME <TIME_VALUE>	{0:1}	p. 107
n LANG <LANGUAGE_OF_TEXT>	{0:1}	p. 94
n **SUBM** <XREF:SUBM>	{0:1}	p. 108
n FILE <GEDCOM_FILE_NAME>	{0:1}	p. 93
n COPR <COPYRIGHT_GEDCOM_FILE>	{0:1}	p. 79
n NOTE <GEDCOM_CONTENT_DESCRIPTION>	{0:1}	p. 93

Form Header

A GEDCOM form does not define a GEDCOM header, it merely defines a form-specific extension to the mandatory <GEDCOM_HEADER>.

Extensions

The Basic GEDCOM header *must not* contain CONC or CONT records.
A form header *may* contain CONC or CONT records, but this is strongly discommended.

The Basic GEDCOM header *must not* contain GEDCOM extensions. A form header *may* contain GEDCOM extensions, but this is strongly discommended.
The form header is complex enough as it is. Developers ignoring this caution should not expect third parties to support their extensions.

Valid Header

It is important that GEDCOM 5.5.5 writers produce correct GEDCOM files, and especially important that each file contains a correct form header. **A GEDCOM 5.5.5 reader must demand a flawless form header; it *must not* process a GEDCOM file if the header is invalid, but must issue a fatal error and abort.**

HEAD.SUBM Optional

GEDCOM 5.5.5 allows just one SUBM record per GEDCOM file, *and* it has to follow immediately after the GEDCOM header. That makes HEAD.SUBM record superfluous.

The HEAD.SUBM is no longer mandatory, but optional now. It may be deprecated in a later version. It is recommended that GEDCOM 5.5.5 writers do create the HEAD.SUBM record, for maximum compatibility with GEDCOM 5.5.1. The HEAD.SUBM pointer is not needed, but when the record is included, the pointer must be valid.

SOUR & DEST

The header structure provides information about the entire GEDCOM file. The SOUR (source) system name identifies which system created the file, and that is all it identifies.
The DEST (destination) system name identifies the intended receiving system.
The HEAD.DEST value identifies the GEDCOM dialects used, i.e. the interpretation of the GEDCOM extension. GEDCOM writers must default the HEAD.DEST value to be equal to the HEAD.SOUR value, to ensure correct interpretation of their system's GEDCOM extensions.

LINEAGE_LINKED_RECORD:=

```
[
n <<FAM_GROUP_RECORD>>                                  {1:1}    p. 58
|
n <<INDIVIDUAL_RECORD>>                                 {1:1}    p. 61
|
n <<MULTIMEDIA_RECORD>>                                 {1:1}    p. 62
|
n <<NOTE_RECORD>>                                       {1:1}    p. 63
|
n <<REPOSITORY_RECORD>>                                 {1:1}    p. 63
|
n <<SOURCE_RECORD>>                                     {1:1}    p. 63
]
```

User-Defined Top-Level Records

GEDCOM allows genealogy software developers to define additional records types, and that includes top-level record types.
For example, multiple developers use a top-level _PLACE or _LOC records to deal with GEDCOM 5.5.1's *place record design error*.

Bold Letters

Most genealogy applications generate cross-reference identifiers that consist of a single capital letter followed by a number, e.g. `@S123@` or `@F456@`. The emboldened letter for each type in the definition above is the suggested letter for records of that type.
A GEDCOM 5.5.5 file contains exactly one <<**S**UBMITTER_RECORD>>, it is suggested that its identifier be `@U1@`.

Reference

Common GEDCOM Identifier Naming Convention

FAM_GROUP_RECORD:=

```
n <XREF:FAM> FAM                                        {1:1}
  +1 <<FAMILY_EVENT_STRUCTURE>>                         {0:M}    p. 67
  +1 HUSB <XREF:INDI>                                   {0:1}    p. 108
  +1 WIFE <XREF:INDI>                                   {0:1}    p. 108
  +1 CHIL <XREF:INDI>                                   {0:M}    p. 108
```

+1 NCHI <COUNT_OF_CHILDREN>	{0:1}	p. 79
+1 REFN <USER_REFERENCE_NUMBER>	{0:M}	p. 107
+2 TYPE <USER_REFERENCE_TYPE>	{0:1}	p. 107
+1 RIN <AUTOMATED_RECORD_ID>	{0:1}	p. 78
+1 <<CHANGE_DATE>>	{0:1}	p. 66
+1 <<NOTE_STRUCTURE>>	{0:M}	p. 71
+1 <<SOURCE_CITATION>>	{0:M}	p. 73
+1 <<MULTIMEDIA_LINK>>	{0:M}	p. 71

The FAM ("family group") record records a *family group*: a couple and their children, if any. The family group record is used to record marriages, common law marriages and other relationships, and all children from that relationship.

relationships

The FAM.MARR record documents the relationship between FAM.HUSB or FAM.WIFE. The nature of the relationship is documented by the optional MARR.TYPE subrecord; if there is no MARR.TYPE record, marriage is assumed.
The following table is a list of the common MARR.TYPE values and their meaning. Developers should use these values for maximum compatability with other applications, and must not use translations, abbreviations or other values that mean the same thing as the values documented here.

unknown	relationship (type unknown)
marriage	marriage
not married	not married
civil	civil marriage
religious	religious marriage
common law	common law marriage
partnership	partnership
registered partnership	registered partnership
living together	living together
living apart together	living apart together

The religious marriage ceremony that may follow a civil marriage is not included in this list, as it does not establish a relationship; it must be recorded as an event.
It would make sense for `unknown` to be the default; `marriage` is the default for backward compatability.

Lineage-Linked

In Lineage-Linked GEDCOM, parents and children are not linked directly to each other, but instead linked through a family group record. A child is linked to its known parent or parents via their family group record, and each family group record links to all known children for those parents.
The preferred order of the CHIL (children) pointers within a FAM (family group) structure is chronological by birth.

The family group record documents unions between at most two people. GEDCOM does not offer any record to document unions of three or more people. Multiple unions and polygamy can and must be recorded by using multiple family group records.

Family Group Record

Each family group record documents just one union between at most two people (one or both may be unknown). Each union, each relationship requires its own family group record. An individual who had more than one relationship, will occur in more than one family group record.
The family group record is the only record for documenting couples, and must be used for all couples.

The HUSB subrecord and WIFE subrecord link to the individuals in the relationship. Despite the gendered names of these subrecords, the family group record can and must be used to document same-sex relationships. It is legal to link the FAM.HUSB record to a female individual, or the FAM.WIFE record to a male individual.

Sex

That the names of HUSB & WIFE subrecords suggest a particular sex and gendered roles is an artefact of GEDCOM's history. The names of these subrecords remain unchanged from previous versions of GEDCOM for backward compatibility, but they should largely be thought of as awkward synonyms for a sex-neutral FAM.SPOU records.
The HUSB & WIFE subrecords do not indicate any sex, gender or role other than partner or parent. The FAM.HUSB will often link to a male individual, and FAM.WIFE will often link to a female individual, but that need not be the case.
Application *must not* assume any particular sex, gender or role based on the FAM.HUSB or FAM.WIFE subrecords.

Sex is specified by INDI.SEX and by INDI.SEX only. Any gendered pronouns used on screen or in reports must be based on the sex documented in the INDI record.

GEDCOM writers must avoid confusing other applications that do still make assumptions about gender or role based on the FAM.HUSB or FAM.WIFE subrecords.
If only a father is known, that father must be linked through the FAM.HUSB record. If only a mother is known, that mother must be linked through the FAM.WIFE record. For any male & female couple, FAM.HUSB must link to the male individual and FAM.WIFE to the female individual. Having the FAM.HUSB record point to the wife, and the FAM.WIFE record to the husband is an error.

Empty Records

There can be at most one HUSB subrecord and one WIFE subrecord per family group record. Neither one is mandatory. Either one can be missing to document one known and one unknown parent for a child of that couple. Both can be missing to document that we know children to be siblings, but do not know who their parents are.
The syntax even allows a family group record without either parents or children. Genealogy applications should not create “empty” family group records, but should automatically delete a family group record when a user has removed all links to individuals. A family group record for just one child makes no sense, but a family group for only a single partner should be maintained, if the MARR subrecord contains information, such as a marriage date (marriage date known, partner unknown).

Remarriage

There may be multiple events for a single relationship, but there may be just one relationship per FAM record. There may be just one relationship per family group record. *Every* new relationship requires a new family group record, even if that relationship is between two people who already had a previous relationship.
This consistent approach keeps things simple and allows other relationships in between.

The family group record allows multiple events of the same type, but they must be for same relationship. This provides the flexibility to record multiple marriage records with different dates and places for the same marriage.
There may be multiple marriages for the same relationship when the legality of one marriage is doubtful or rejected by another jurisdiction. There may also be multiple marriage records, with different dates and places, for a single marriage, for example when a Dutch couple marries in Germany and then goes back to their home town in the Netherlands, the German town will create a marriage record, and the Dutch town will create a marriage record several days or weeks later, when the couple comes back and shows the town clerk that they've been married.